Comparison Between Pseudocount and Soft-Imputation Left-censored Zero Handling

Christian Anderson, 31 October 2026

Abstract. **Background:** Compositional data, such as gut metagenomic relative abundances, lie on a simplex and contain spurious correlations that analyses assuming Euclidean geometry do not account for, so they are usually transformed with the centered log-ratio (CLR). The CLR is undefined at zero, yet biological compositions are sparse (often more than 50% zeros), and these zeros are often due to technical effects such as limited sampling depth (left-censoring). They must therefore be replaced before the transformation. **Methods:** I compared replacing zeros with a small constant (a pseudocount) to imputing them with SVD soft-imputation. I simulated Dirichlet-multinomial count data with a group effect on a subset of features and tuned each method over a grid of its parameters. I then compared sensitivity (power) at a false discovery rate (FDR) below (or at least near) 0.05 from per-feature t-tests on the CLR-transformed data. **Results:** At a fold change of 5, the best pseudocount (3.3×10^{-5}) had a sensitivity of 0.17 at an FDR of 0.04, and the best soft imputation settings ($\lambda = 0.12$, rank fraction $= 0.90$) had a sensitivity of 0.15 at an FDR of 0.052. Across fold changes from 0 to 10, the sensitivity of both methods rose and plateaued around 0.2 by $\text{fc} \approx 5$ and FDR stayed at or below about 0.05 for both. **Conclusions:** In this sparse simulation with few differential features, soft imputation gave no advantage in sensitivity or FDR over a tuned pseudocount, so the hypothesis was not supported. Power was low for both methods in these initial tests, but further comparisons are needed at different parameter settings, specifically testing with more differential features and varying sparsity. **Keywords.** Compositional Data, Zero Imputation, Centered Log-Ratio, Dirichlet-Multinomial

Introduction

Compositional data lie on the simplex, meaning values are between 0 and 1 and sum to 1 within each observation; when one feature rises, others must fall so spurious negative correlations can arise. Transforming the data from the simplex (Aitchison space) to Euclidean space avoids this and allows standard tools such as the t-test and PCA. The most common transformation is the centered log-ratio (CLR), which centers log abundances on their geometric mean[1]:

$$\text{clr}(x_j) = \log(x_j) - \text{mean}(\log(x))$$

Biological compositional data are also sparse: most features are zero in the majority of samples. A zero means the feature was either absent or below the detection limit of the technical process; we assume the latter (left-censoring) and treat zeros as missing values. Because $\log(0)$ is undefined, these zeros must be replaced before the CLR.

Motivation

Of the many zero-replacement methods, two are simple and widely used. A pseudocount replaces zeros with a small constant, often $\frac{\min(x_{x \neq 0})}{2}$, although the choice may strongly affect downstream results. Soft imputation instead iteratively computes the singular value decomposition (SVD) of the data, fills the zero cells from a low-rank reconstruction, and repeats until the reconstruction stops changing. By shrinking the singular values each time, only the most informative components determine the imputed values. Soft imputation has more parameters to tune, and it may introduce artificial structure for individual features. A pseudocount is simple but adds artificial variance. The inferior method would add noise and inflate false positives.

Explored Question

Across fold changes from 0 to 10, does soft imputation give higher sensitivity (power) than a tuned pseudocount when the false discovery rate (FDR) is below 0.05 for per-feature group tests on CLR-transformed data? *I hypothesize that soft imputation preserves the multivariate structure better than a pseudocount and will therefore show higher sensitivity at FDR < 0.05.*

Simulation Setup

To test which method is better, I simulate sparse compositional data and introduce a known effect. I simulate with the zeros built in instead of adding them artificially, as this is more representative of biological data. The simulation step also uses many parameters, so I use a grid search to identify the best ones for each method.

Generative Model. Counts are modeled by sampling probability vectors, θ, from a Dirichlet distribution which is each sample's true composition[2]. For each sample, I sample a Multinomial N_i times from that sample's probability vector, θ_i, simulating a realistic sequencing depth library size which varies across samples. This step also introduces zeros by chance and the fixed library size means counts are negatively correlated by construction (as a feature that gets more draws leaves fewer for others). The library sizes N are sampled from a Log-Normal with a log-scale mu (median) and sd (multiplicative). I then convert the simulated counts to relative abundance by dividing each sample's counts by its total counts, yielding P. See supplamental material for modeling scheme (**Figure S1**).

Parameters. I simulated $n = 100$ samples (50 per group) and $p = 5000$ features. The mean composition m was drawn from a log-normal distribution (log-scale mean 0, sd 1.5) and normalized to sum to 1, and the Dirichlet parameter was $\alpha = m \cdot \text{concentration}$. The concentration controls the variation between samples, that is, how tightly samples cluster around m[3,4]. Without concentration, features were more variable and sparse. Library sizes N_i had a median of 10^6 reads and a log-scale sd of $\frac{\ln(2)}{2}$, a 2-fold range. I clipped these values to $[10^5, 10^7]$. Features that were zero in at least 90% of samples were removed, roughly 700 features.

Effect. For samples in group A, the probability θ_{ij} of each differential feature was multiplied by the fold change fc and each θ_i was renormalized to sum to 1; group B was unchanged. The differential features were a random 1% of features (50) in both the grid searches,

run at an arbitrarily chosen $\mathrm{fc} = 5$, and the sweep over fold changes. This effect increases group A features and the renormalization shifts all other features down slightly as a result.

Zero handling. For the pseudocount, every zero in the relative abundances P was replaced by a constant c and nonzero entries were left unchanged. For soft imputation, I first converted values to the log-scale and then set zeros to half the smallest observed value of their feature. Each iteration then centered the columns, took the SVD, shrank the singular values by λ times the largest one, kept the top r components (r = rank fraction $\times \min(n, p)$), and replaced the zeros with the reconstruction. Iteration stopped when the change in the sum of squared imputed log-values fell below 10^{-3} or after 100 iterations. Both methods were followed by the CLR transform.

Testing and evaluation. For each feature I ran a two-sample t-test between the groups on the CLR values, adjusted the p-values with the Benjamini-Hochberg procedure and labeled features with $q < 0.05$ as discovery. Because the differential features are known, I computed sensitivity $= \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$, specificity $= \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$, and FDR $= \frac{\mathrm{FP}}{\mathrm{TP} + \mathrm{FP}}$. In the grid searches these were averaged over five simulation seeds, and the best setting of each method was the one with the highest sensitivity among settings with FDR < 0.05 or the lowest FDR (when none are below 0.05). The two methods were then compared at their best settings across 100 fold changes from 0 to 10, with one simulated dataset per fold change (**Table 1** summarizes the settings).

Setting	Value
Samples n, features p	100 (50 per group), 5000
Mean composition	log-normal(0, 1.5), normalized
Zero fraction	0.5
Library size N_i	log-normal, median 10^6, sd $\frac{\ln(2)}{2}$, clipped to $[10^5, 10^7]$
Prevalence filter	features with $\geq 90\%$ zeros removed
Differential features	1%
Fold change	5 (grid search), 0 to 10 (sweep)
Pseudocount grid	30 values, 10^{-8} to 10^{-2} on log-scale
λ grid	10 values, 0.01 to 0.9 on log-scale
Rank fraction grid	10 values, 0.01 to 0.9 on log-scale
Seeds	120, 100, 113, 1111, 245

Table 1: Simulation settings.

Results

Lambda	Rank Fraction	Specificity	Sensitivity	FDR
0.122	0.900	1.000	0.149	0.052

Table 2: Best soft imputation settings.

Pseudocount	Specificity	Sensitivity	FDR
3.29e+−5	1.000	0.171	0.042

Table 3: Best pseudocount (highest sensitivity with FDR < 0.05).

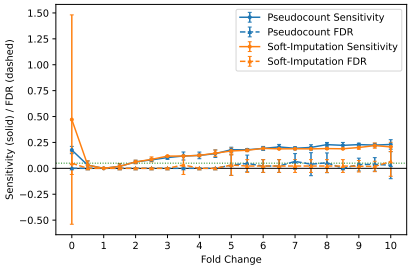


Figure 1: Sensitivity (solid) and FDR (dashed) of the pseudocount and soft imputation versus fold change. The curves overlap.

Few settings resulted in an FDR < 0.05 for either soft imputation (**Figure S2**) or pseudocount (**Figure S3**), and there does not appear to be a relationship between these parameters and performance for either (Spearman partial-correlation $q > 0.05$). At $\mathrm{fc} = 5$, the optimal pseudocount was 3.3×10^{-5}, which achieved a sensitivity of 0.17 with FDR < 0.05 (**Table 3**), and the optimal soft imputation parameters were $\lambda = 0.122$ and rank fraction $= 0.90$, which had a sensitivity of 0.15 but an FDR just over the threshold (0.052; **Table 2**). Across 100 fold changes between 0 and 10, the sensitivity and FDR curves of the two methods are nearly indistinguishable (**Figure 1**). Both metrics increased as fc moved away from 1 for both methods, and at $\mathrm{fc} = 1$, where there is no difference between groups, neither method detects any features beyond the nominal false positive rate (as expected).

Discussion

Soft imputation did not improve sensitivity over a tuned pseudocount, so the hypothesis was not supported. Both methods had low power at $\mathrm{fc} = 5$ (0.15 to 0.17), and their sensitivity and FDR curves were indistinguishable across fold changes. A likely reason is that the methods differ only in the values given to zero cells, and the differential features detected in the tuning runs were abundant features without zeros. Any remaining difference enters only through each sample's CLR geometric mean, which shifts all features in a sample together and does not change the group comparison unless the shift differs between groups. I did not test this explanation directly. With 1% of features differential, the choice of zero handling therefore did not matter for detecting a group difference, and a simple tuned pseudocount is sufficient.

Limitations

Both methods were tuned at a single fold change ($\mathrm{fc} = 5$) and applied unchanged across all fold changes, so the settings may not be best at other effect sizes. I used one data-generating model, with zeros arising only from sampling and treated as left-censored, at one sparsity level and one effect fraction (1%), and I tuned and evaluated on the same simulation model. Real zeros may also be structural or technical, which would change the results. Next steps are to vary the fraction of differential features, the sample size and number of features, the concentration (sparsity), and the zero mechanism, or to use a Gamma-Poisson model, which gives a similar structure with fewer parameters.

References

1. Hron, K., Templ, M. & Filzmoser, P. Imputation of missing values for compositional data using classical and robust methods. *Comput. Stat. Data Anal.* **54,** 3095–3107 (2010).
2. Zamzami, N. & Bouguila, N. A Novel Scaled dirichlet-based statistical framework for count data modeling: Unsupervised learning and exponential approximation. *Pattern Recognit.* **95,** 36–47 (2019).
3. 21.2 Reparameterizations (https://mc-stan.org/docs/2_26/stan-users-guide/reparameterizations.html#dirichlet-priors).
4. Dirichlet mixtures of multinomials (https://docs.pymc.io/projects/examples/en/latest/mixture_models/dirichlet_mixture_of_multinomials.html).

Supplemental Materials

$$
\begin{aligned}
X_i &\sim \text{Multinomial}(N_i, \theta_i) \\
\theta_i &\sim \text{Dirichlet}(\alpha) \\
\alpha &= m * \text{concentration} \\
m &\sim \text{lognormal}(0, 1.5) \\
N_i &\sim \text{lognormal}(\mu, \sigma) \\
P_{ij} &= \frac{X_{ij}}{\sum_{k=1}^{p} X_{ik}} \\
\text{where } X \in \mathbb{N}^{n \times p}, &\quad i = 1, ..., n, \quad \text{and} \quad j = 1, ..., p
\end{aligned}
$$

Figure S1: Generative Model.

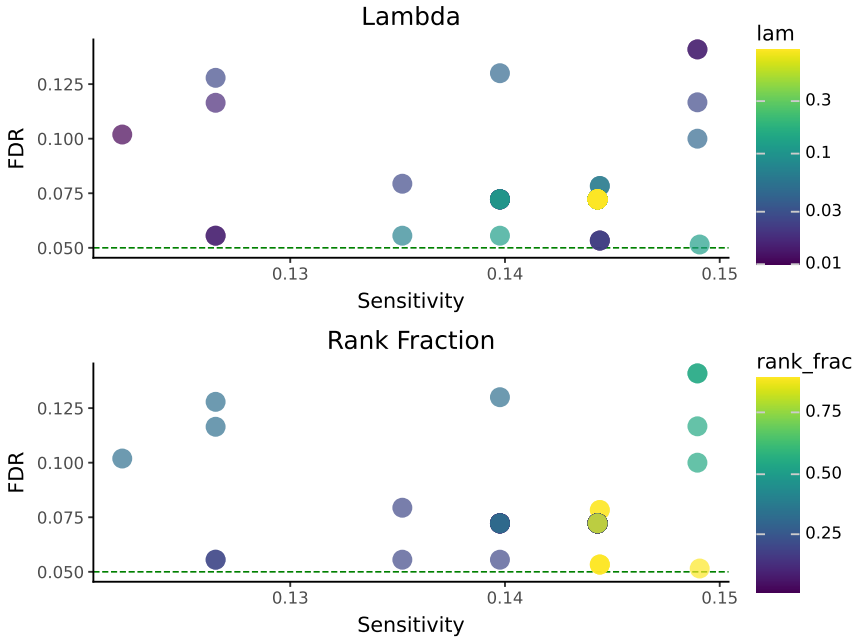


Figure S2: Soft imputation grid search over λ and rank fraction (mean over five seeds). Each point is one setting; the dashed line is FDR = 0.05.

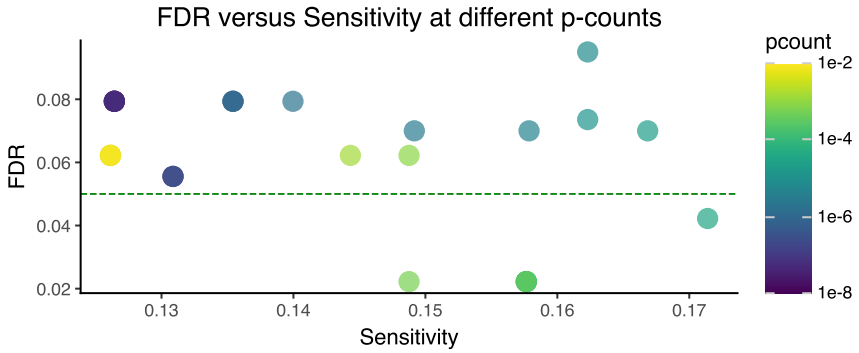


Figure S3: Pseudocount grid search (mean over five seeds). Each point is one pseudocount; the dashed line is FDR = 0.05.

Appendix

Code

Code is available on my **Github**.

```python
1
2 # ——— code/impute.py
3 import numpy as np
4
```

```python

def soft_impute(X, W, maxit=50, rank=10, lam=.1, tol=1e-3, early_stop=True, cap=True):
    """Impute masked entries of X with iterative soft-thresholded SVD in log space.
    Masked entries start at half the smallest observed value in their column. each iteration then centers the log matrix by column, takes the SVD, shrinks singular values by lam * (max(singlr.val.)), keeps the top `rank`, and reconstructs. Observed entries are never changed.
    Parameters:
        X: (n, p) non-negative data matrix.
        W: (n, p) boolean mask, True = missing/to impute.
        maxit: max iterations. rank: max singular values kept.
        lam: shrinkage as a fraction of the largest centered singular value (0 = none, 1 = everything removed).
        tol: stop when the change in delta is below this if early_stop is True (delta is in log units).
        early_stop: stop when difference between iterations is < tol.
        cap: cap imputed log values at the column's smallest observed log value (a zero is below detection).
    Returns:
        (delta, X_imputed):  where delta is the squared change in imputed log values per iteration.
    """
    def _reconstruct(L):
        mu = L.mean(0, keepdims=True)
        u, s, vt = np.linalg.svd(L - mu, full_matrices=False)
        s_ = np.maximum(s - lam * s[0], 0)
        return mu + (u[:, :rank] * s_[:rank]) @ vt[:rank]
    logX = np.log(np.where(W, 1, X))
    col_min = np.nanmin(np.where(W, np.nan, logX), axis=0)  # log of the smallest observed value per column
    Z = _reconstruct(np.where(W, col_min - np.log(2), logX))  # initial fill with min - log(2) -> min/2
    if cap:
        Z = np.minimum(Z, col_min)
    delta = []
    for _ in range(maxit):
        prev = Z
        Z = _reconstruct(np.where(W, Z, logX))  # imputed where 0, original elsewhere
        if cap:
            Z = np.minimum(Z, col_min)
        delta.append(np.sum((prev[W] - Z[W]) ** 2))
        if early_stop and len(delta) > 1 and np.abs(delta[-1] - delta[-2]) < tol:
            break
    else:
        if len(delta) > 1:
            print(f'No convergence to `tol` at {maxit} iterations')
    return delta, np.where(W, np.exp(Z), X)

# ——— code/main.py
from re import L
import numpy as np
from plotnine import *
from pipeline import run, sweep
import os
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd
from dataclasses import dataclass
from tqdm import tqdm
```

```python
from scipy import stats


if __name__ == '__main__':
    os.chdir(Path.home() / 'Documents/school/BIOS6011/assignments/midterm')

    # \\\
    # \\\
    # \\\
    fc = 5
    seeds = [120,100,113,1111,245]

    # \\

    # ——— Tune p-count
    # \\

    pcounts = np.geomspace(1e-8, 1e-2, 30)
    RES_p = sweep('pcount', {'pcount': pcounts}, fc=fc, seeds=seeds )

    p = (
        ggplot(data = RES_p, mapping = aes(x = 'sens', y = 'fdr'))
        + geom_hline(yintercept = 0.05, linetype = '--', color = 'green')
        + geom_point(aes(color = 'pcount'), alpha = .7, size = 5)
        + scale_color_continuous(trans = 'log10')
        + labs(title="FDR versus Sensitivity at different p-counts", x = 'Sensitivity', y = 'FDR')
        + theme_classic()
    )
    p.save('out/pcount_sens_fdr_scatter.pdf', height = 2.5, width = 6)

    optim_p = RES_p.loc[RES_p['fdr'].lt(.05)].sort_values("sens", ascending = False).iloc[0]
    optim_p.to_frame().T.to_csv("out/optim_p.csv", index = False)

    # \\

    # ——— Tune imp
    # \\

    lams = np.geomspace(.01, .9, 10)  # fraction of the largest singular value
    ranks = np.geomspace(0.01, .9, 10)
    RES_i = sweep('imp', {'lam': lams, 'rank_frac': ranks}, fc=fc, seeds=seeds)

    def panel(var, trans = 'identity'):
        lab = ''
        if var == 'lam':
            lab =  "Lambda"
        elif var == 'rank_frac':
            lab =  "Rank Fraction"
        return (
            ggplot(data = RES_i, mapping = aes(x = 'sens', y = 'fdr'))
            + geom_hline(yintercept = 0.05, linetype = '--', color = 'green')
            + geom_point(aes(color = var), alpha = .7, size = 5)
            + scale_color_continuous(trans = trans)
            + labs(title = lab, x = 'Sensitivity', y = "FDR")
            + theme_classic()
```

```
        )
    p = panel('lam', 'log10') / panel('rank_frac')
    p.save('out/imp_sens_fdr_scatter.pdf', height = 5, width = 6)

    optim_i = RES_i.loc[RES_i.fdr.eq(RES_i.fdr.min())].sort_values("sens", ascending = False).iloc[0]
    optim_i.to_frame().T.to_csv("out/optim_i.csv", index = False)

    X = RES_i.select_dtypes("number")
    corr = X.corr(method="spearman")
    PREC = np.linalg.inv(corr)
    D_inv = np.diag(1 / np.sqrt(np.diag(PREC)))
    partial_corr = pd.DataFrame(-D_inv @ PREC @ D_inv, index=X.columns, columns=X.columns)

    n, k = len(X), X.shape[1] - 2
    df = n - 2 - k

    # table over unique pairs only (upper triangle)
    iu = np.triu_indices(X.shape[1], k=1)
    r = partial_corr.values[iu]
    t = r * np.sqrt(df / (1 - r**2))
    res = pd.DataFrame({"var1": X.columns[iu[0]], "var2": X.columns[iu[1]],"r": r, "p": 2 * stats.t.sf(np.abs(t), df),})
    res["q"] = stats.false_discovery_control(res.p, method="bh")   # scipy >= 1.11
    res["signif"] = res.q < .05
    res.sort_values("p").round(3)

    # \\

    # ——— Plot fdr/sens vs fc for both
    # \\
    # \\
    sens_fdrs = []
    FCs = np.linspace(0,10,100)

    for fc in tqdm(FCs):
        s, delta, res, scores =  run(fc = fc, pcount = optim_p.pcount, col = 'q', lam = optim_i.lam, rank_frac = optim_i.rank_frac, \
            seed = seeds, verbose = False)
        sens_fdrs.append(scores[['sens', 'fdr']].assign(fc = fc))
    sens_fdrs_d = pd.concat([x.reset_index() for x in sens_fdrs], axis = 0)

    raw = sens_fdrs_d.copy()
    raw['fc_group'] = (raw['fc']*2).round()/2

    plt.figure(figsize = (6, 4))
    colors = {'pcount': 'C0', 'imp': 'C1'}
    styles = {'sens': ('-', 'o'), 'fdr': ('--', '^')}
    for meth in ['pcount', 'imp']:
        dm = raw.loc[raw['index'].eq(meth)]
        for metric, (ls, mk) in styles.items():
            g = dm.groupby('fc_group')[metric].agg(['mean', 'std', 'count']).dropna()   # drops groups with NaN/1 run
            half = stats.t.ppf(.975, g['count'] - 1) * g['std'] / np.sqrt(g['count'])  # 95% CI half-width
            plt.errorbar(g.index, g['mean'], yerr = half, linestyle = ls, marker = mk,
```

```
                        markersize = 3, capsize = 2, color = colors[meth], label = f"{meth} {metric}")
    plt.axhline(.05, color = 'green', linestyle = ':', linewidth = 1)   # nominal FDR
    plt.axhline(0, color = 'black', linestyle = '-', linewidth = 1)   # nominal FDR
    plt.ylabel("Sensitivity (solid) / FDR (dashed)")
    plt.xlabel("Fold Change")
    plt.xscale("linear")
    plt.xticks(range(FCs.min().astype(int), FCs.max().astype(int)+1, 1))
    plt.legend()
    plt.tight_layout()
    # plt.show()
    plt.savefig("out/sens_fdr_fc.pdf")


    print("*** Optimal parameters ***")
    print(f"lambda = {optim_i.lam:.3f}, rank fraction = {optim_i.rank_frac:.3f}")
    print(f"pcount = {optim_p.pcount:.3e}")
    s, delta, res, scores =  run(fc = .0001, pcount = optim_p.pcount, col = 'q', lam = optim_i.lam, rank_frac = optim_i.rank_frac, \
        seed = np.arange(100), effect_frac = .1, verbose = False)
    scores
    scores[['tp', 'fp', 'fn', 'tn', 'sens', 'spec', 'fdr']].round(2).to_csv("out/table.csv")

    d = sens_fdrs_d.reset_index(drop = True)
    d = d[d.fdr < .05]

    d.loc[d.groupby("index").sens.idxmax()].rename({'index':'method'}, axis= 1).to_csv("out/best_sens_under_fdr.csv")

    optim_i

# ——— code/pipeline.py
import itertools
import os

import numpy as np
import pandas as pd
from multiprocess import cpu_count, get_context

from sim import simulate
from impute import soft_impute
from process import clr, standardize, test_features, metrics

METRICS = ['spec', 'sens', 'fdr']


def run(fc=3, lam=.1, pcount=1e-5, rank_frac=.2, maxit=100, tol=1e-3, col='p', alpha=.05, verbose=True, **sim_kw):
    """Simulate, impute zeros two ways, test each feature, and score.
    fc and any other simulate() argument (n, p, zero_target, alpha_sigma, ...) go to the simulation.
    lam, rank_frac, maxit, tol configure soft_impute; pcount replaces the zeros (only the zeros) in the comparison.
    col is 'p' or 'q' and alpha the cutoff used as feature detected.
    Returns (sim, delta, results, scores).
    """
    s = simulate(fc=fc, **sim_kw)
```

```
    W = s.P == 0
    delta, imp = soft_impute(s.P, W, rank=int(min(s.P.shape) * rank_frac), maxit=maxit, tol=tol, lam=lam)
    if verbose:
        print(f"Imputation: iter = {len(delta)}; SSE = {delta[-1]:.2e}; median imputed zero = {np.median(imp[W]):.2e}")
    res = {
        'imp': test_features(standardize(clr(imp)), s.groupA, s.is_effect),
        'pcount': test_features(standardize(clr(np.where(W, pcount, s.P))), s.groupA, s.is_effect),
    }
    scores = pd.DataFrame({k: metrics(v, col, alpha) for k, v in res.items()}).T
    return s, delta, res, scores


def pmap(fn, jobs, n_jobs=None):
    """Map fn over jobs in worker processes."""
    n_jobs = n_jobs or max(cpu_count() - 1, 1)
    if n_jobs == 1:
        return [fn(j) for j in jobs]
    names = ["OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "MKL_NUM_THREADS", "VECLIB_MAXIMUM_THREADS"]
    old = {k: os.environ.get(k) for k in names}
    os.environ.update({k: "1" for k in names})  # inherited by workers started below
    try:
        with get_context("spawn").Pool(n_jobs) as pool:
            return pool.map(fn, jobs, chunksize=1)
    finally:
        for k, v in old.items():
            if v is None:
                os.environ.pop(k, None)
            else:
                os.environ[k] = v


def _task(args):
    method, fc, fixed, kw = args
    *_, scores = run(**{'fc': fc, 'col': 'q', 'verbose': False, **fixed, **kw})
    return scores.loc[[method], METRICS].assign(**kw)


def sweep(method, grid, fc=3, seeds=range(5), n_jobs=None, **fixed):
    """Score one method over a grid of arguments and seeds in parallel.
    method: 'imp' or 'pcount'.
    grid:  e.g. {'lam': lams, 'rank_frac': ranks}.
    fixed: other run() arguments held constant (calls default to col='q').
    Returns METRICS averaged over seeds, one row per grid point.
    """
    keys = list(grid)
    jobs = [(method, fc, fixed, {**dict(zip(keys, vals)), 'seed': seed})
            for seed in seeds for vals in itertools.product(*grid.values())]
    out = pd.concat(pmap(_task, jobs, n_jobs))
    return out.groupby(keys, as_index=False)[METRICS].mean()

# ——— code/process.py
import numpy as np
import pandas as pd
```

```
import scipy.stats as stats


def kde(x,r = None,**kwargs):
    if r is None:
        min,max  = x.min(), x.max()
    else:
        min,max = r
    range = np.linspace(min,max,500)
    return range,stats.gaussian_kde(x, **kwargs)(range)

def clr(X):
    L = np.log(X)
    return L - L.mean(1, keepdims = True)

def standardize(X):
    return (X - X.mean(0, keepdims = True)) / X.std(0, keepdims = True)

def test_features(Z, groupA, is_effect):
    """
    two-sample t-test of group A vs B on the columns of Z.
    Returns:
        DataFrame with t, p, BH q, mean difference a-b and the truth label.
    """
    t, p = stats.ttest_ind(Z[groupA], Z[~groupA], axis=0)
    return pd.DataFrame({
        't': t,
        'p': p,
        'q': stats.false_discovery_control(p, method='BH'),
        'a-b': Z[groupA].mean(0) - Z[~groupA].mean(0),
        'has_effect': is_effect,
    })

def metrics(res, col='p', alpha=.05):
    """
    Confusion metrics.
    """
    det, eff = (res[col] < alpha).to_numpy(), res['has_effect'].to_numpy()
    tp, fp = (det & eff).sum(), (det & ~eff).sum()
    fn, tn = (~det & eff).sum(), (~det & ~eff).sum()
    div = lambda a, b: a / b if b else np.nan
    return {'tp': tp, 'fp': fp, 'fn': fn, 'tn': tn,
            'sens': div(tp, tp + fn), 'spec': div(tn, tn + fp),
            'fdr': div(fp, tp + fp), 'ppv': div(tp, tp + fp), 'npv': div(tn, tn + fn)}

# ——— code/sim.py
from dataclasses import dataclass

import numpy as np
from scipy.optimize import brentq


@dataclass
class Sim:
    X: np.ndarray            # (n, p_retained) counts
    P: np.ndarray          # (n, p_retained) relative abundances
```

```
    is_effect: np.ndarray     # (p_retained,) truth labels, aligned with columns of X
    groupA: np.ndarray        # (n,) True for samples with the effect
    N: np.ndarray          # (n,) library sizes
    zero_frac: float       # zero fraction before the sparsity filter
    alpha_concentration: float


def zero_frac(conc, m, N_med):
    """Gamma-Poisson approximation of the mean P(X=0) across features."""
    return np.mean((1 + N_med / conc) ** (-m * conc))


def simulate(n=100, p=5000, alpha_sigma=1.5, zero_target=0.5, alpha_concentration=None,
    N_median=1e6, N_sigma=np.log(2)/2, N_clip=(1e5, 1e7), fc=3, effect_frac=0.01, max_zero=0.9,
    seed=21233):
    """Simulate an n x p count matrix with a group effect on a subset of features.
    m ~ Lognormal(0, alpha_sigma), normalized: mean composition.
    alpha = m * alpha_concentration; if alpha_concentration is None it is solved so the expected
    zero fraction is zero_target.
    theta ~ Dirichlet(alpha) per sample; group A's theta on effect features is multiplied by fc,
    then renormalized. N ~ Lognormal(log(N_median), N_sigma), clipped to N_clip.
    X ~ Multinomial(N, theta). Features with zero fraction >= max_zero are dropped.
    is_effect is all False when fc == 1 (null simulation).
    """
    rng = np.random.default_rng(seed)
    m = rng.lognormal(mean=0, sigma=alpha_sigma, size=p)
    m /= m.sum()
    if alpha_concentration is None:
        alpha_concentration = brentq(lambda c: zero_frac(c, m, N_median) - zero_target, 1, 1e6)
    alpha = m * alpha_concentration
    theta = rng.dirichlet(alpha, size=n)
    feats = rng.choice(p, int(round(p * effect_frac)), replace=False)
    groupA = np.arange(n) < (n / 2)
    theta[np.ix_(groupA, feats)] *= fc
    theta /= theta.sum(1, keepdims=True)
    N = rng.lognormal(mean=np.log(N_median), sigma=N_sigma, size=n).clip(*N_clip).astype(int)
    X = rng.multinomial(N, theta).astype(float)
    zf = (X == 0).mean()
    is_effect = np.zeros(p, bool)
    if fc != 1:
        is_effect[feats] = True
    keep = (X == 0).mean(axis=0) < max_zero
    X, is_effect = X[:, keep], is_effect[keep]
    P = X / X.sum(axis=1, keepdims=True)
    return Sim(X, P, is_effect, groupA, N, zf, alpha_concentration)

```